

Name: Jared Rosenblum

Date: June 4, 2026

Course: IT FDN 110 GP – Foundations of Python Programming

Github Repository: <https://github.com/JR-3/IntroToProg-Python-Mod07>

Assignment 07 - Creating a Python Program

Introduction

This document describes the steps taken to create a Python program which provides the user with a menu of options to register a student for a course. The program allows the user to register students by inputting the student's first and last name, along with the course they're registering for. As long as this option is chosen, the user can continue to register as many students as needed. The user can then review the registration data and/or save it to an enrollment .json file. When finished, there is an option for the user to exit the program. Additionally, all of the registration data previously entered will still be available to review upon exiting and/or rerunning the program.

Creating and Executing the Python Script

Before creating the Python program, an Integrated Development Environment (IDE) is recommended to assist in writing, testing, managing and debugging the underlying code.¹ One of the more intuitive IDE's is PyCharm, which can be downloaded and installed at the following link - <https://www.jetbrains.com/pycharm/download>.

Once PyCharm is installed and a new project and .py file are created, the following steps can be taken to develop and execute a script that will display a menu with options to begin inputting individual student registration details, view the data that's been entered, save it to an enrollment .json file, and exit the program.

1. Begin by importing the JSON module which is a built-in module in Python. JSON is short for JavaScript Object Notation and is a lightweight data interchange format that is easy for people to read and write, and is also easy for machines to parse and generate. The files consist of key-value pairs organized in { } for objects and [] for arrays.² The module can be imported by writing the syntax *import json* at the beginning of the program.
2. Define and set constants to be used as part of the program, which will remain unchanged.³ The constants will be a list of menu options displayed to the user when they go through and run the program. This includes four options to 1) Register a student for a course, 2) Show current data, 3) Save data to a file and 4) Exit the program. Additionally, FILE_NAME is set up to indicate the name of the .json file that will be written to and saved down as part of menu option 3.

As a standard convention to help identify a constant in the script, it should be written in uppercase letters.⁴ The constants used as part of the script are as follows:

```
# Define the Data Constants
MENU: str = ''
---- Course Registration Program ----
Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
-----
'''
# Define the Data Constants
FILE_NAME: str = "Enrollments.json"
```

3. Define and set global variables to be used throughout the program - “students” and “menu_choice.” The “students” variable, a list, would hold one row of the student data input by the user, which would encompass the students first and last name and course that they’re registered for. By using the input() function as part of the “menu_choice” variable, the user will be prompted to select one of the options from the menu.

```
# Define the Data Variables
students: list = [] # a table of student data
menu_choice: str = '' # Hold the choice made by the user.
```

4. To help organize the code, follow the Separation of Concerns (SOC) design principle, by dividing the program’s functionality into distinct and isolated concerns. A concern is a specific aspect or responsibility of a program’s functionality. Three primary layers found in such architectures are the data layer, processing layer, and presentation layer. The data layer is responsible for handling data storage and retrieval, including working with constants and variables. The processing layer is responsible for processing and transforming the data as required by the application and is also responsible for enforcing security and managing error handling. The presentation layer, also known as the input/output layer, focuses on creating the visual and interactive elements of an application that the user is presented with along with capturing their input(s).⁵ The use of both classes and functions can be used to assist in following the SoC principle.
 - **Classes** are a fundamental concept in programming that provide a way to organize code by grouping functions, variables, and constants by the name of that class.⁶
 - For the purpose of the program, set up a *Person* class, *Student* class (to inherit the *Person* class), *FileProcessor* class and *IO* class. This would address the data layer, processing layer and presentation layer, respectively.

- **Functions/Methods**, which are grouped under specific classes, are a reusable block of code that performs a specific task or set of tasks. They help manage larger programs by making code easier to understand, maintain, and debug. Additionally, once a function is defined, its code can be used multiple times throughout the program.⁷
 - As part of the *Person* and *Student* class (data layer), add a special method called a constructor (using the syntax `__init__`) to initialize the underlying object's attributes. Constructors assign initial values to the object's data attributes, ensuring that it starts with a defined state. Attributes are variables that hold data specific to an object.⁸
 - As part of the constructor, create special functions called properties to both *set* and *get* each attribute's value. A *getter* property function enables you to access data and apply formatting. A *setter* property function allows you to add validation and error handling.⁹
 - As part of the *FileProcessor* class, define the *read_data_from_file* and *write_data_to_file* functions.
 - As part of the *IO* class, define the *output_error_message*, *output_menu*, *input_menu_choice*, *input_student_data*, and *output_student_courses* functions.

As part of each of the functions, create parameters that provide data to each function when called. Parameters help make clear what the function needs to operate correctly and eliminates the reliance on global variables, which are defined outside the function and are global to the entire script.¹⁰

5. Create *Person* Class and related constructor and properties

Create a new class called *Person* to handle the data associated with a student's first name and last name. Using the syntax `__init__` create a constructor class that takes the *first_name* and *last_name* parameter, defaulting the values to empty strings. Define both the *first_name* and *last_name* attributes as part of the constructor.

Add a getter property function using `@property` to access the data and apply formatting for both *first_name* and *last_name* attributes. Add a matching setter property function for both *first_name* and *last_name* to add in validation and error handling.⁹

As part of convention, add the keyword *self* throughout the syntax, that refers to whichever object you are currently instantiating.¹¹

```

# TODO Create a Person Class
class Person:

# TODO Add first_name and last_name properties to the constructor
    def __init__(self, first_name: str = '', last_name: str = ''):
        self.first_name = first_name
        self.last_name = last_name

# TODO Create a getter and setter for the first_name property
    @property # (getter or accessor)
    def first_name(self):
        return self.__first_name.title() # formatting code

    @first_name.setter
    def first_name(self, value: str):
        if value.isalpha() or value == "": # is character or empty string
            self.__first_name = value
        else:
            raise ValueError("The last name should not contain numbers.")

# TODO Create a getter and setter for the last_name property
    @property
    def last_name(self):
        return self.__last_name.title() # formatting code

    @last_name.setter
    def last_name(self, value: str):
        if value.isalpha() or value == "": # is character or empty string
            self.__last_name = value
        else:
            raise ValueError("The last name should not contain numbers.")

# TODO Override the __str__() method to return Person data
    def __str__(self):
        return f'{self.first_name},{self.last_name}'

```

6. Create *Student Class* and related constructor and properties

Create a new class called *Student* to inherit the code from the *Parent* class, along with setting up the methods and properties for the name of the course that a student is registered for.

The constructor (`__init__`) of the *Student* class is extended to include an additional parameter, `course_name`, representing the name of the course that a student is registered for. It uses `super().__init__` to call the constructor of the parent *Person* class, which initializes the `first_name` and `last_name` attributes.¹²

```

# TODO Create a Student class the inherits from the Person class
class Student(Person):

# TODO call to the Person constructor and pass it the first_name and last_name data
# TODO add a assignment to the course_name property using the course_name parameter
    def __init__(self, first_name: str = '', last_name: str = '', course_name: str = ''):
        super().__init__(first_name=first_name, last_name=last_name)
        self.course_name = course_name

# TODO add the getter for course_name
    @property # (getter or accessor)
    def course_name(self):
        return self.__course_name.title() # formatting code

# TODO add the setter for course_name
    @course_name.setter
    def course_name(self, value: str):
        self.__course_name = value

# TODO Override the __str__() method to return the Student data
    def __str__(self):
        return f'{self.first_name},{self.last_name},{self.course_name}'

```

7. *Read_data_from_file* function (part of FileProcessor class):

Define a function named *read_data_from_file* and set the *file_name* and *student_data* as its parameters.

In order to extract and read the Enrollment.json file to work with, use the *open()* function, with the “r” option. This accesses the contents of the file without modifying them.¹³ The *json.load(file)* method parses the JSON data from *FILE_NAME* into a Python list of dictionaries.¹⁴

Add additional syntax to convert the list of JSON dictionary objects (*json_students*) to the newly created student objects (consisting of *first_name*, *last_name*, and *course_name*) that are being used throughout the program.¹⁵

Surrounding the code is a structured error handling method called try-except. Error handling improves the script by managing errors that you may not have control over any other way. You can actively plan for mistakes and make the process of recovering from errors smoother. The Try-except construct is an organized way to trap errors, while providing general and user friendly error messages.¹⁶

In the case of opening the file, a try-except block contains the Exception class and *output_error_message* function (set up in step 9 below) to help address any errors. The finally block runs after the try and except blocks finish, ensuring the file is always closed properly.¹⁷

```

@staticmethod
def read_data_from_file(file_name: str, student_data: list):

    file = None

    try:
        # Get a list of dictionary rows from the data file
        file = open(file_name, "r")
        json_students = json.load(file)

        # Convert the list of dictionary rows into a list of Student objects
        #student_objects = []
        # TODO replace this line of code to convert dictionary data to Student data
        #student_objects = json_students
        for student in json_students:
            student_object: Student = Student(first_name=student["FirstName"],
                                                last_name=student["LastName"],
                                                course_name=student["CourseName"])
            student_data.append(student_object)

        file.close()

    except Exception as e:
        IO.output_error_messages(message="Error: There was a problem with reading the file.", error=e)

    finally:
        if file is not None and file.closed == False:
            file.close()

    return student_data

```

8. *Write_data_to_file* function (part of FileProcessor class):

Define a function named *write_data_to_file* and set the *file_name* and *student_data* as its parameters.

Since JSON cannot directly handle custom objects like instances of the newly created student class, convert the data in student objects back into a dictionary format that is compatible with JSON. For each object, create a dictionary that is appended to a list called *json_students*.¹⁸

Write the content of the students table by first opening the *FILE_NAME* (Enrollments.json) file in write mode using the *open()* function. Include a "w" as part of the code to indicate that you want to write to the file.¹⁹

Then using the *json.dump()* function, write the first name, last name, and course name data captured as part of the students table into the file.²⁰ Finally, close the file using the *close()* function.¹⁹

To help handle errors, surround the code with the try-except construct. Use the Exception class and *output_error_message* function (set up in step 9 below) to handle any errors that could arise from going through and closing the file.¹⁶

The finally block runs after the try and except blocks finish, ensuring the file is always closed properly.¹⁷

```
@staticmethod
def write_data_to_file(file_name: str, student_data: list):

    file = None

    try:
        # TODO: Add code to convert Student objects into dictionaries (Done)
        json_students: list = []

        for student in student_data:
            student_json: dict \
                = {"FirstName": student.first_name, "LastName": student.last_name, "CourseName": student.course_name}
            json_students.append(student_json)

        file = open(file_name, "w")
        json.dump(json_students, file, indent=2)
        file.close()
        #json.dump(student_data, file)
        IO.output_student_and_course_names(student_data=student_data)

    except Exception as e:
        message = "Error: There was a problem with writing to the file.\n"
        message += "Please check that the file is not open by another program."
        IO.output_error_messages(message=message, error=e)

    finally:
        if file is not None and file.closed == False:
            file.close()
```

9. *Output_error_message* function (part of IO class):

Define a function named *output_error_messages* and set message in the form of a string and error in the form of an exception as its parameters. The following function is set up to display a custom error message to the user. A technical error message can also be displayed, with the attribute `error.__doc__` providing a brief description of the exception type and what it signifies. The `type(error)` syntax will print the type of exception object.¹⁶

```
class IO:

    @staticmethod
    def output_error_messages(message: str, error: Exception = None):
        print(message, end="\n\n")
        if error is not None:
            print("-- Technical Error Message -- ")
            print(error, error.__doc__, type(error), sep='\n')
```

10. *Output_menu* function (part of IO class):

Define a function named *output_menu* and menu as its parameter.

By using the `print()` function, display the menu data constant set up as part of step 2 above to the user.

```
# Present the menu of choices
@staticmethod
def output_menu(menu: str):
    print(menu)
```

11. *Input_menu_choice* function (part of IO class):

Define a function named *input_menu_choice* to prompt the user to enter menu option 1, 2, 3, or 4.

Similar to previous functions, surround the code with the try-except construct to help handle errors, including the *output_error_message* function set up previously.

```
@staticmethod
def input_menu_choice():
    choice = "0"
    try:
        choice = input("Enter your menu choice number: ")
        if choice not in ("1", "2", "3", "4"):
            raise Exception("Please, choose only 1, 2, 3, or 4")
    except Exception as e:
        IO.output_error_messages(e.__str__())
    return choice
```

12. *Output_student_and_course_names* function (part of IO class):

Define a function named *output_student_and_course_names* and set *student_data* as its parameter.

Use the *print()* function to display the student set up previously.

Concatenate the data entered using an f-string (introduced as part of Python 3.6). An f-string allows you to embed expressions directly inside string literals by prefixing the string with "f." In this case, attributes set up as part of the *Person* and *Student* classes are accessed and used as expressions inside curly-braces {}. These are then evaluated and their values are inserted into the string.²⁴

The *print("-"*50)* is for presentation purposes only, displaying dashes around the data.


```

@staticmethod
def output_student_and_course_names(student_data: list):

    print("-" * 50)
    for student in student_data:
        print(f'Student {student.first_name} '
              f'{student.last_name} is enrolled in {student.course_name}')
    print("-" * 50)

```

13. *Input_student_data* function (part of IO class):

Define a function named *input_menu_choice* and set *student_data* as its parameter.

Set up the function to allow the user to input a student's first name, last name, and course they're registered for. Using the attributes set up as part of the Person and Student class, add the data to an object called student. Each student registration is then appended to a list called *student_data*.

As part of the *print()* function, concatenate the data entered using an f-string (introduced as part of Python 3.6).²⁴ This will print a message letting the user know that the student that had just registered.

A try-except error handling method is used. Both the *ValueError* and *Exception* classes, along with the *output_error_message* function, are used to catch and handle specific input errors on the student first name and last name, as well as any other exceptions. The *e.__doc__* syntax attribute will provide a brief description of the exception type and what it signifies. The *type(e)* syntax will print the type of exception object.¹⁶

```

@staticmethod
def input_student_data(student_data: list):

    try:
        student = Student()
        student.first_name = input("Enter the student's first name: ")
        student.last_name = input("Enter the student's last name: ")
        student.course_name = input("Please enter the name of the course: ")
        student_data.append(student)
        print()
        print(f"You have registered {student.first_name} {student.last_name} for {student.course_name}.")

    except ValueError as e:
        IO.output_error_messages(message="One of the values was the correct type of data!", error=e)
    except Exception as e:
        IO.output_error_messages(message="Error: There was a problem with your entered data.", error=e)
    return student_data

```

14. Referencing the classes and defined functions, build out the core structure of the program. Read the data from the Enrollments JSON file and then cycle through a “while loop” which displays the menu to the user, asking them to select one of the four menu options. The while loop will execute a set of instructions repeatedly as long as a certain condition remains true.²¹ Based on the syntax, the program will continuously display the menu and related options - input student data, display current student data, save new student data, and exit the program - for the user to select. It’s important to note that the loop will run indefinitely unless a break is included as part of the syntax to end the loop prematurely.²² In this case, option 4 will allow the user to exit the program via the break statement. To control the flow of the program, inside the while loop, create conditional “if” and “elif” statements associated with each of the menu options.²³

```
# Present and Process the data
while True:

    # Present the menu of choices
    IO.output_menu(menu=MENU)

    menu_choice = IO.input_menu_choice()

    # Input user data
    if menu_choice == "1": # This will not work if it is an integer!
        students = IO.input_student_data(student_data=students)
        continue

    # Present the current data
    elif menu_choice == "2":
        IO.output_student_and_course_names(student_data=students)
        continue

    # Save the data to a file
    elif menu_choice == "3":
        FileProcessor.write_data_to_file(file_name=FILE_NAME, student_data=students)
        continue

    # Stop the loop
    elif menu_choice == "4":
        break # out of the loop
    else:
        print("Please only choose option 1, 2, or 3")

print("Program Ended")
```

The following is the menu displayed for the user, along with the results of selecting each of the four options:

Menu option 1: allows user to register a student for a course:

```
Enter your menu choice number: 1
Enter the student's first name: Tim
Enter the student's last name: Thomas
Please enter the name of the course: Python 400
```

Menu option 2: displays data that has been entered by the user:

```
Enter your menu choice number: 2
-----
Student Vic Vu is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student Vic Vu is enrolled in Python 3000
Student Marshawn Lynch is enrolled in Python 5000
Student Sue Bird is enrolled in Python 6000
Student Gary Payton is enrolled in Python 7000
Student Tim Thomas is enrolled in Python 7500
-----
```

Menu option 3: saves data to Enrollments.json file:

Name	Date modified
 Assignment07.py	6/4/2026 9:33 AM
 Enrollments.json	6/3/2026 3:27 PM

Menu option 4: ends program

```
Enter your menu choice number: 4
Program Ended

Process finished with exit code 0
```

Entering an option other than 1, 2, 3 or 4

```
Enter your menu choice number: 11
Please, choose only 1, 2, 3, or 4
```

Summary

By creating the Python program outlined in steps 1-14 above, a user will be presented with a menu of options upon running the program. The program allows the user to input a student's first and last name, along with the course they're registering for (option 1). As long as the first menu option is chosen, the user can continue to register as many students as needed. The user can then review the registration data (option 2) and save it to a .json file (option 3). When finished, there is an option for the user to exit the program (option 4). All of the registration data that was input will be saved to the .json file and continue to be available for review upon exiting and/or rerunning the program.

References

1. MOD03-Notes.pdf page
2. MOD05-Notes .pdf page 13
3. MOD01-Notes.pdf page 28
4. MOD01-Notes.pdf page 32
5. Mod06-Notes pgs. 32-33
6. Mod06-Notes p. 27
7. Mod06-Notes p.2-3
8. Mod07-Notes pgs. 6-7
9. Mod07-Notes pg. 15
10. Mod06-Notes p.14-15
11. Mod07_notes pg. 8
12. Mod07-Notes p.24
13. MOD04-Notes.pdf page 29
14. MOD05-Notes .pdf page 17
15. Mod07-Notes p. 29
16. MOD05-Notes .pdf pages 19-20
17. MOD05-Notes .pdf page 24
18. Mod07-Notes p. 30
19. MOD02-Notes.pdf pages 17-18
20. MOD05-Notes .pdf page 16
21. MOD03-Notes.pdf page 16
22. MOD03-Notes.pdf page 17
23. MOD03-Notes.pdf page 8
24. MOD02-NOTES.pdf page 9